CSCI5527 DEEP LEARNING PROJECT
A LITERATURE REVIEW OF THE EVOLUTION OF CNNS AND THEIR APPLICATION FOR AUDIO CLASSIFICATION

Alex Ojemann, Jathin Mallampati & Sarthak Bansal
Department of Computer Science and Engineering
University of Minnesota
Minneapolis, MN 55455

1 ABSTRACT

Convolutional Neural Networks (CNNs) have evolved to become the dominant model architecture for image classification, especially since the graphics processing unit (GPU) revolution of the late 2000s and early 2010s. Eventually CNNs were so successful in image classification that they spread to other tasks, particularly the classification of audio. This paper chronicles the evolution of the architectures of these CNNs as well as the methods used to train them, beginning with the earliest CNNs trained on CPUs, their rapid expansion once GPUs became widely used in training and larger datasets became more widely available, and culminating in today's hybrid convolution-attention systems trained with self-supervised objectives on millions of audio clips. We distill how pooling, batch normalization, residual links, and depth-wise separable kernels lifted accuracy across speech, music, scene, and speaker tasks. Efficiency advances now permit real-time inference on earbuds and IoT mics. Remaining challenges include data efficiency, robustness to noisy environments, and interpretability of learned filters.

2 INTRODUCTION

Convolutional Neural Networks (CNNs) are a class of deep learning models designed to learn spatial or temporal trends of features that could not be captured by standard fully connected neural networks. Rather than being composed of only fully connected layers, CNNs have mostly convolutional and pooling layers in the beginning of the network. Convolutional layers apply a kernel matrix, sometimes alternatively referred to as a filter matrix, in a sliding window across a two dimensional image or other two dimensional representation, multiplying the kernel matrix by the input values in the sliding window detecting spatial or temporal patterns that would not be detected in a fully connected network. Pooling layers achieve a similar objective by aggregating the input values in the sliding window. Critically, CNNs can be used for any application in which the spatial and temporal trands in the data need to be considered, so even though they were initially used strictly for image classification, their use has also been extended to audio classification in recent years.

Section 3.1 of this paper discusses the earliest application of a deep CNN for image classification. Section 3.2 discusses the advancements facilitated by the use of GPUs in training and the methodological changes that came with this. Section 3.3 discusses AlexNet Krizhevsky et al. (2012), the first CNN to achieve state-of-the-art results on a truly large-scale image classification task. Section 3.4 discusses the advancements in the use of CNNs immediately following AlexNet that cemented the status of CNNs as the best models for large-scale image classification. Section 3.5 discusses the initial uses of the CNN architectures in section 3.4 for audio classification.

3 BODY

3.1 EARLY YEARS

Before 2000, the foundational concepts of CNNs had been established, but there were some significant obstacles to their use in image classification at scale. LeNet-5 Lecun et al. (1998), often credited as the first truly deep CNN, was relatively small by today's standards and was limited to only using 5x5 convolutional kernels but showed superior results to several other existing models for classification of handwritten digits such as RBF networks, fully connected neural networks, tangent distance classifiers, and support vector machines with nonlinear kernels. In addition to its superior accuracy, it was more purely data driven than the methods to which it was compared in that it did not require manual augmentations to the training images such as deslanting, cropping, manual selection of the kernel function, and additional domain-specific augmentations. The use of convolutional average-pooling layers eliminated the need for these practices; essentially, the features were learned by the network rather than manually engineered. However, there was still one key element of the model that had to be completed manually, which was the creation of target prototype vectors for each class. These prototype vectors represented the expected output from the second to last layer of the network of a perfect representation of one of the output classes. The last layer was formed by taking the euclidean distance from the output of the second to last layer to each of the target prototype vectors for each output class, and the loss function was the mean squared error between the output of the last layer and the correct class label. For the purposes of the MNIST handwritten digits dataset on which this network was trained, this was not exceedingly time-consuming because there are only 10 output classes, one for each digit, so only 10 target prototype vectors had to be created. However, for larger-scale image classification problems, there are many more output classes, so this is not practical. This network is also constrained in its ability to generalize to larger-scale problems by the vanishing gradient problem. The use of the hyperbolic tangent activation function makes this network particularly sensitive to the vanishing gradient problem because its derivative is very small for all inputs other than those near zero. In addition, this network was also very difficult to train due to the use of a single CPU architecture. The authors note that it took multiple days to train, making it extremely impractical for use on larger datasets. The use of a quasi second-order optimization method, rather than a first order method, also contributed to its lengthy training time. In the ensuing decade, some additional improvements were made in the LeNet-5 architecture and training methods to achieve better accuracy on the MNIST dataset with less required training time Simard et al. (2003), but image classification tasks that were more complex than classifying handwritten digits remained largely dominated by architectures that used handcrafted feature extraction and a shallow classifier Dalal & Triggs (2005) Lowe (2004) Viola & Jones (2001).

3.2 GPU REVOLUTION

In the late 2000s and early 2010s, a series of developments drastically improved the performance of CNNs and allowed them to surpass that of the models using handcrafted feature extraction techniques that previously dominated large scale image classification problems. Ciresan et al. (2011) created the first network to be trained with GPU acceleration and achieve state-of-the-art performance on the MNIST dataset. The main advancement that led to their success was the recent development of Nvidia's CUDA software, which dramatically increased the potential for developers to utilize the parallel processing capability of GPUs to train neural networks. Before CUDA, there were very few software tools to facilitate the training of neural networks using GPUs as they were primarily used for rendering graphics. The authors explicitly note that the biggest difference between their network architecture and that of LeNet-5 is the use of max-pooling layers in between convolutional instead of average-pooling, as max pooling had separately been shown to result in both faster convergence and improved generalization Scherer et al. (2010). Though otherwise architecturally similar to LeNet-5, their network was much more computationally demanding to train because it used many more feature maps, as many as 420 maps across seven hidden layers compared to just 44 maps across seven hidden layers for LeNet-5, resulting in orders of magnitude more learnable weights. The memory demands of these added feature maps on the low-memory GPUs used in training, along with other concerns regarding parallelism and the CUDA software, mandated several changes to the methods used to train the network. The first was the use of stochastic gradient descent (SGD), a first-order method, in training rather than the much more memory-heavy quasi-second order methods used to train LeNet-5. Critically, the authors found that the quality of each individual

update step of SGD was not substantially worse than those of quasi-second order methods in terms of getting to the solution, despite requiring much less memory and compute per step. The switch from quasi second-order methods to first-order methods would prove to be an enduring change in the course of evolution of all neural networks. Along with the switch to first-order methods methods came the use of the cross-entropy loss function the softmax function for calculating output probabilities, in contrast to the RBF target prototype vectors and mean squared error loss of LeNet-5. The cross-entropy loss and softmax combination had been shown to have a uniquely simple derivative relative to the outputs of the network before the last layer for each class z_i, which is $\hat{y}_i - y_i$ Bridle (1990). This is very easy to compute as opposed to the euclidean distance calculation required for the last layer of LeNet-5 and its calculation is much more numerically stable. Additionally, there was a small but critical change to how gradients were backpropagated through the convolutional layers to help facilitate parallelism. While LeNet-5 used a strategy known as "pushing deltas" which requires iterating through each neuron in the current layer and passing its gradient to each neuron in the previous layer from which it receives contributions. In contrast, this network used a strategy known as "pulling deltas" which requires iterating through each node in the previous layer and determining which nodes in the current layer are contributed to by that node and pulling the gradients from them. Pushing deltas does not require the inversion of the convolution operation to find which outputs are contributed to by each input and thus was preferable for CPU-based CNNs like LeNet-5. However, if pushing deltas were to be run in parallel, it would require multiple threads, each of which is responsible for one or more neurons in the current layer, writing to the same neuron in the previous layer, which would require either the use of additional memory buffers or the use of much slower methods of updating that prevent multiple threads from writing to each neuron at the same time. Pulling deltas avoids this conundrum because each of these nodes in the previous layer is being processed by a distinct thread. To be clear, the authors were not the first to use each one of these techniques, but they were the first to put them together to achieve state-of-the-art performance on MNIST as well as two other benchmark image classification datasets, NORB and CIFAR-10, which amounted to a monumental step forward for CNNs. However, the MNIST, NORB and CIFAR-10 datasets only had 10, five, and 10 output classes respectively, so additional improvements would have to be made before CNNs could solve larger-scale image classification problems.

3.3 ALEXNET

AlexNet Krizhevsky et al. (2012) built on the innovations discussed in the previous section and made several improvements to the state-of-the-art CNNs at the time that finally enabled CNNs to overtake models using handcrafted feature extraction techniques on large-scale classification problems. This network was trained on the ImageNet Large Scale Visual Recognition Challenge dataset (ILSVRC) which contains 3.2 million images and 1000 different classes, both orders of magnitude larger than the MNIST, NORB, and CIFAR-10 datasets. A few innovations that may seem small but had enormously beneficial effects on performance and computation allowed this network to achieve state-of-the-art performance on ILSVRC. The first of these was the augmentation of the dataset before the network was trained. The authors added translations, horizontal reflections, and differing color intensities of the original images, which made the network substantially more robust to data outside the training set. The authors used a novel technique to generate the images with differing color intensities which involved performing PCA on the 3x3 covariance matrix of the red, green, and blue values, creating perturbation vectors of length three by scaling each principal component vector by the eigenvalues and a small random perturbation coefficient, and then adding these perturbation vectors to each pixels original red, green, and blue values to get the perturbed image, effectively perturbing it along the three most important orthogonal directions of its color scheme. Secondly, the authors recognized that the hyperbolic tangent activation function was contributing to the vanishing gradient problem and switched to a ReLU activation. The ReLU activation has a derivative of 0 or 1 for all inputs, which is much better than the frequent very small but nonzero derivative values of the hyperbolic tangent function for the purposes of numerical stability in backpropagation. This proved to be an enduring innovation not just for CNNs, but for almost all neural networks going forward because of its broad applicability. Along with the ReLU activation function came the local response normalization technique, which effectively downweights the activation of neurons which have neighbors with high activations. This was implemented partially because the ReLU activation function has no upper bound, unlike the scaled hyperbolic tangent used in previous networks, so it prevented excessively high activations for some neurons, but it also served to prevent neurons from learning the same features which significantly helped the network generalize to new data.

Thirdly, the authors used dropout regularization to prevent overfitting. Overfitting was of particular concern because the added images from data augmentation were heavily correlated with all of the others that were based on the same original image. The authors' dropout strategy involved randomly dropping hidden neurons from the network with probability 0.5 during training, which significantly reduces the co-adaptation of neurons Hinton et al. (2012). The authors explicitly noted that when trained without dropout regularization, their network exhibited substantial overfitting. Since this is a stochastic process in which different neurons are dropped in each batch, the authors note that they must use all neurons when testing the network but multiply their outputs by 0.5 to mimic the effect of dropping half of them. Fourthly, the authors used varying sizes of convolutional kernel across layers. The first convolutional layer had a kernel size of 11x11 and a stride length of four, the second convolutional layer had a kernel size of 5x5 and a stride length of one, and the third, fourth, and fifth convolutional layers had kernel sizes of 3x3 and stride lengths of 1, whereas LeNet-5 and other previous networks had the same kernel size and stride length throughout the network. The idea behind these kernel sizes and stride lengths is that the first one reduces the dimensionality significantly to limit the memory and compute burden for lower layers, but the subsequent layers do not do so to nearly as high of a degree to mimic human vision by increasing complexity with decreasing size. Finally, the authors were particularly innovative in their usage of GPUs. They used two GPUs and only shared previous layer inputs between them in the layers for which this was most optimal. They compared this to an approach in which only one GPU and half as many kernels were used and the dual-GPU approach was significantly more accurate with slightly less required training time.

3.4 Further Improvements in Large-Scale Image Classification

After AlexNet set the stage, many other researchers tried to build upon its success on the ILSVRC dataset with modifications to its architecture and training. One of the networks which succeeded in doing so was VGG Simonyan & Zisserman (2015). The authors explicitly stated that their network was inspired by many of the principles of both the network developed by Ciresan et al. (2011) and AlexNet specifically. However, there was one critical difference; they only used 3x3 convolutional kernels and strides of length one. This was accompanied by more layers in the network in order to sufficiently reduce the resolution before feeding the outputs into a feed forward classifier. AlexNet featured larger kernels and stride lengths in the earlier layers as described in the previous subsection, and Ciresan et al. (2011) used similarly small kernels and stride lengths but was trained on much lower resolution images in datasets with far fewer output classes than ILSVRC. The logic behind this is that multiple layers of 3x3 kernels will achieve equivalent reduction of the resolution with fewer parameters than a single layer of any larger kernel size. An example the authors point to is that three 3x3 kernels and one 7x7 kernel equally reduce the resolution but the 3x3 kernels only require $27C^2$ parameters while the 7x7 kernel requires $49C^2$ parameters where C is the number of channels for both the input and the output. The authors use a similar training procedure to that of AlexNet as well as similar data augmentation prior to training, but one key difference was the pre-initialization of certain layers of the network. The authors initially trained a smaller network, then used the weights from some of its layers as initial values for those in certain layers of subsequent layers to help facilitate convergence. This practice was particularly useful for this network because it was substantially deeper than its predecessors. The authors showed that VGG not only achieved state-of-the-art performance on ILSVRC, but also generalized well to a variety of other classification tasks.

The next key development for CNNs was reflected by the GoogLeNet Szegedy et al. (2014) and Inception Szegedy et al. (2016) networks. GoogLeNet was created for the same competition as VGG and yielded a very similar result in one of the two commonly used error metrics for the ILSVRC dataset. Their most significant amendment to the architecture of AlexNet was inspired by the theoretical work of Arora et al. (2013), which regarded the development of an algorithm for layer-wise construction of a CNN in which similar neurons in each layer are clustered together. The creators of GoogLeNet built the network to mimic the process of clustering similar neurons without explicitly running a clustering algorithm in each layer, which can be very computationally expensive for large datasets. The network featured 1x1, 3x3, and 5x5 convolutional kernels within each layer, in contrast to AlexNet and other previous architectures which only used one kernel size within each layer. This more flexible approach effectively gave the network the ability to learn which of the kernel sizes were optimal within each layer for different portions of a given image. Allowing the network to learn differing weights for kernels of differing sizes mimicked the process of creating

different sizes of clusters based on which neurons are most similar. The creators of GoogLeNet also used included a 3x3 max pooling operation within the same layer as the convolutional kernels, whereas AlexNet and other previous networks used max pooling layers in between convolutional layers further increasing the ability of the network to learn different patterns within different regions of the image. Padding was used for the feature maps created 3x3 and 5x5 convolutional kernels and the 3x3 max pooling operation so that these would all be the same size as the feature maps created by the 1x1 kernel to input to the next layer. Since this meant that these layers would not reduce the resolution of the input, max pooling layers with stride length two were added between some of them, each of which would halve the width and height of the input. Inception, which was created by some of the same creators of GoogLeNet, built on the success of GoogLeNet in three key ways. First was the factorization of the 5x5 kernels into two separate 3x3 kernels, which, as shown by the creators of VGG and discussed in the previous paragraph, resulted in fewer parameters but was functionally very similar. Secondly, batch normalization, the normalization of all layer outputs before activation in a given training mini-batch, was used. In a separate paper, two of the creators of Inception showed that batch normalization eliminated the shift of the distribution of each layer's inputs during training, which significantly sped up training and eliminated the concerns regarding initial values articulated by the creators of VGG, and also showed that it can serve as a regularizer and eliminate the need for dropout Ioffe & Szegedy (2015). The efficiency gains from the factorization of 5x5 kernels and batch normalization allowed the creators to extend the network to 42 layers deep, much deeper than any of the previous networks discussed, but with just 2.5 times the computational cost of GoogLeNet. Finally, they made their resolution reduction layers more efficient. For resolution reduction, GoogLeNet simply had max pooling layers with stride length two, whereas Inception had multiple convolution and pooling operations run in parallel and concatenated similar to its other layers which did not reduce dimensionality, which is much better for preserving complex trends while still achieving an equal reduction in resolution. These improvements allowed Inception to achieve meaningfully better performance on ILSVRC than both GoogLeNet and VGG.

The final key development for CNNs in the immediate aftermath of AlexNet was reflected in the network ResNet He et al. (2016). The creators of ResNet completely rethought the objective of a CNN. Instead of having the entire CNN learn the entire function $H(x)$, they suggested that instead the network could learn the residual function $F(x)$, where $F(x) = H(x) - x$, with x defined as the original input. The way this is implemented is through shortcut connections. A shortcut connection is a connection between layers in a network that skips over intermediate layers. They were used to bring the input values, x, to the other side of the layers that learned $F(x)$, and then $F(x)$ and x could be added together to get $H(x)$. This structure could be repeated many times to develop a powerful network where each block learned the residual of the previous block. ResNet's architecture largely imitates that of VGG with almost all layers being 3x3 convolutional kernels but with many of these blocks that learn the residual of the previous block facilitated by a shortcut connection from the beginning of the block to the end. Since this practice was found to facilitate much more efficient learning than that of previous networks, the largest version of the network contained 152 layers, which previously would have been unfathomable to train. Though the architecture resembles that of VGG much more so than that of Inception, the creators did use batch normalization. This resulted in state-of-the-art performance on ILSVRC and multiple other datasets.

These critical innovations served as a basis for many future improvements in the use of CNNs in a wide variety of applications. Understanding the techniques described up to this point is essential to understanding the applications discussed in the rest of this paper and many others.

3.5 Initial Applications for Audio Classification

Researchers began to apply CNNs to audio classification after the creation of AlexNet, but it was not until Hershey et al. (2017) that all of the architectures in the previous subsection were applied to the task and compared to a one dimensional fully connected classifier. The paper primarily focused on classifying the soundtracks of 100 million Youtube videos labelled by their genre, for which there are 30,871 distinct labels and each soundtrack can have multiple labels. The process for turning the one dimensional audio from each Youtube video into a two dimensional spectrogram involves segmenting the audio into fixed windows, converting to a spectrogram using a Short Term Fourier Transform, separating the frequencies into 64 bins and taking a log transformation of the value within the bins. This is known as a log-mel spectrogram. Its dimensions are 96x64, much smaller than the 224x224 images that were inputs to the CNN architectures used, so modifications were

made to the initial layers of the CNNs to account for this. Predictions for each video soundtrack were calculated by averaging the probabilities from each spectrogram within the video soundtrack. Though there were multiple labels per video, each label was treated independently, so AUC, an error metric based on true positives, true negatives, false positives, and false negatives, could be used in evaluation. Of the four CNN architectures used, AlexNet, VGG, Inception, and ResNet, Inception scored the best in terms of AUC when the training was limited to the same number of mini-batches for all models, but all four architectures significantly outperformed the baseline fully connected classifier. Additionally, the authors used the feature embeddings from the convolutional and pooling layers of the models to train a classifier on the AudioSet benchmark dataset. AudioSet contains an ontology of hundreds of sound classes and over 2 million labeled 10-second clips Gemmeke et al. (2017). The classifier trained on the feature embeddings performed substantially better than a classifier trained on the raw data. In total, this represented a monumental step forward for the application of cutting-edge CNN architectures to audio classification.

3.6 CNN Architectures and Input Representations

A crucial preprocessing step in CNN-based audio classifiers is converting 1D audio waveforms into 2D time–frequency images that CNNs can readily analyze Zaman et al. (2023). Directly using raw waveform samples as 1D input to CNNs is possible (and we discuss such 1D CNN approaches later), but most successful models rely on spectrogram representations. Common choices include the short-time Fourier transform (STFT) spectrogram, mel-frequency spectrogram (log-mel spectrogram), and mel-frequency cepstral coefficients (MFCCs) Zaman et al. (2023). These representations encode the distribution of energy across frequency bands over time, effectively turning audio into an "image" that CNNs can process with 2D convolutional kernels. For example, a mel-spectrogram uses the mel scale for its frequency axis, which is spaced according to human pitch perception. Figure 2 shows a sample spectrogram where time flows along one axis and frequency along the other, with intensity indicating amplitude Zaman et al. (2023). By using such 2D inputs, CNNs exploit spatial locality to learn features like harmonics, formant patterns, or onsets that are characteristic of different sounds.

In practice, the raw waveform is windowed into short frames (e.g. 20–40 ms) with overlap, each frame transformed via FFT to a frequency spectrum, and these spectra are stacked over time to form a 2D array Zaman et al. (2023). Many systems apply a mel filterbank and log amplitude compression to produce a log-mel-spectrogram, which has become a de facto standard input for audio CNNs. Some works use MFCCs (which are essentially a cosine transform of the log-mel spectrum) as input channels to a CNN, although CNNs generally perform better with raw spectrograms than with the heavily compressed MFCC features. More recent research explores learnable front-ends: for instance, SincNet (2018) introduced parametric sinc-convolutions as the first layer to learn an optimal filterbank directly from raw waveforms Ravanelli & Bengio (2019). SincNet's learned band-pass filters gave improved efficiency and accuracy in speaker and speech recognition compared to standard CNNs on raw waveforms Ravanelli & Bengio (2019).

Just as spectrograms translate 1-D audio into 2-D images, DeepInsight maps any feature vector into an image where related dimensions are spatially adjacent (Sharma et al. (2019)). This illustrates why CNNs have begun to appear in domains far beyond sight or sound. DeepInsight Sharma et al. (2019) takes this a step further, generalizing the conversion to image data by taking the non-image data in the form of feature vectors and transforming it into images for a CNN, where this image puts similar features spatially close together and dissimilar features farther apart. DeepInsight was trained on several very different kinds of datasets, specifically one for gene expression, a speech dataset for vowels, a text dataset, and two artificial datasets. DeepInsight outperformed state-of-the-art random forest models for each of these datasets in terms of classification accuracy by an average of around 9%. Furthermore, because there are more well-documented ways to vectorize audio data than other non-image data, audio classification stands out from other non-image classification tasks as something CNNs can be used to improve on.

1D, 2D, and 3D CNN Variants: Early audio CNN models borrowed architectures from image recognition. Hershey et al. showed that analogs of AlexNet, VGG, and Inception can all yield high audio tagging performance when trained on massive data Hershey et al. (2017). These are 2D CNNs operating on spectrogram inputs (treating the time–frequency image as a 2D spatial input). A 2D CNN will convolve across time and frequency dimensions in each layer, capturing local patterns like frequency harmonics or temporal modulations. By stacking many convolutional layers with pooling, a

deep 2D CNN can learn hierarchical audio features – from low-level spectral edges up to high-level patterns distinguishing, say, a dog bark from a car horn. For example, the VGGish model (proposed by Hershey et al.) is a VGG-style CNN pre-trained on YouTube audio; its 128-dimensional embedding features have been widely used as audio descriptors Hershey et al. (2017). Another family of models are ResNet-based CNNs, which introduce skip connections; these have been adapted for audio classification (e.g. for acoustic scenes and speaker recognition) due to their strong performance and ease of training with very deep layers.

In parallel, purely convolutional models that operate on 1D raw waveforms have been investigated. 1D CNNs slide filters over the time axis of the waveform directly, learning to extract features without an explicit spectrogram. Notable examples include Facebook's wav2letter (2016) and NVIDIA's Jasper (2019), which are end-to-end speech recognition models composed of stacked 1D convolutions. Jasper, in particular, used a deep 54-layer temporal CNN and achieved then-state-of-the-art word recognition on LibriSpeech (2.95% WER on test-clean) using only convolution, batch norm, and ReLU layers Li et al. (2019). The success of Jasper demonstrated that with sufficient depth and data, 1D CNNs can model long-range temporal dependencies in speech as effectively as recurrent networks. However, 1D models often require more data or layers to match the performance of 2D spectrogram-based CNNs, because they must learn the frequency decomposition from scratch. Hybrid approaches like Wavegram (Kong et al., 2020) combine raw-waveform 1D convolution in early layers with spectrogram-based 2D convolution in later layers, leveraging both representations Kong et al. (2020).

A less common variant are 3D CNNs, which can be applied when an extra dimension is present in the data (for example, multi-channel audio or time-series of spectrogram frames). In audio, 3D convolutions have seen limited use; one example is in spatiotemporal audio processing where a 3D CNN might convolve across time, frequency, and channel (e.g. left vs. right stereo) dimensions. Some sound event detection systems treat the spectrogram as a sequence of "frames" and use a 3D CNN to capture a temporal window of frames in one convolutional block. Generally, though, sequence modeling in audio is handled by recurrent layers or attention mechanisms rather than by very deep 3D conv layers. Still, 3D CNNs have been explored for video audio analysis (e.g. combining audio and visual streams), and for music transcription where one might convolve across time, frequency, and musical pitch dimensions. In summary, modern audio classification CNNs predominantly use 2D convolution on time–frequency inputs, sometimes augmented with 1D front-ends or followed by sequence modeling layers. Next, we examine how these CNN architectures are applied in various audio classification domains, noting task-specific architectures and techniques.

3.7 CNNs in Major Audio Classification Tasks

3.7.1 Speech Recognition (ASR) with CNNs

In automatic speech recognition, CNNs have been employed both in hybrid DNN-HMM systems and end-to-end models. Abdel-Hamid et al. (2012) were among the first to show CNNs can improve phoneme recognition by exploiting frequency locality (using convolution over frequency bins) Zaman et al. (2023). Throughout the late 2010s, CNNs became standard in acoustic modeling, often replacing or complementing fully-connected layers. For example, the popular Time-Delay Neural Network (TDNN) used in Kaldi's "x-vector" speaker models is essentially a 1D convolution over frames (time delays) that produces robust speech embeddings.

Several end-to-end ASR architectures based purely on CNNs emerged around 2016–2019. Facebook's wav2letter demonstrated that a stack of temporal convolutions with gating could directly map speech waveforms to text sequences. NVIDIA's Jasper (Just Another Speech Recognizer) further proved the efficacy of deep CNNs for speech Li et al. (2019). Jasper consists of blocks of 1D convolutions and achieved a word error rate of 2.95% on LibriSpeech test-clean (with an external language model) – a record at the time Li et al. (2019). Its success showed CNNs can handle long-range temporal context needed for transcribing speech. Subsequently, smaller CNN-based models like QuartzNet used factorized 1D convolutions (separable kernels) to reduce parameter count while retaining accuracy, making on-device ASR feasible.

The recent state-of-the-art in ASR has trended toward transformer-based sequence models, but even these often include CNN components. A prime example is the Conformer (Convolution-augmented Transformer). Conformer integrates a convolutional module within each transformer block to cap-

ture local feature patterns, combined with global self-attention Gulati et al. (2020). This hybrid achieved a further 15% relative WER improvement on LibriSpeech over a pure self-attention model. The convolution in Conformer acts as a learnable front-end that improves the model's efficiency and accuracy by providing inductive bias for locality. Thus, even in the transformer era, CNNs remain vital in speech recognition systems – often as front-ends or as parts of multi-stream architectures (e.g. combining a CNN branch and a self-attention branch).

3.7.2 Environmental Sound & Acoustic Scene Classification

Environmental sound classification (ESC) involves recognizing everyday sounds (dog barks, sirens, rain, etc.), and acoustic scene classification involves labeling the scene or environment (park, restaurant, train station, etc.) from an audio recording. These tasks often have limited data (on the order of tens of hours at most), so CNN models need to be efficient and robust to overfitting. CNNs became popular for ESC after Piczak (2015) applied a simple 2D CNN to spectrograms and achieved strong results on the ESC-50 dataset. In the last few years, deeper CNNs and data augmentation have pushed accuracy on ESC-50 (which has 2000 clips in 50 classes Piczak) well beyond human accuracy (81%). Architecture Adaptations: Many ESC systems use architectures inspired by ImageNet models. For instance, a VGG-like CNN or a ResNet-50 applied to mel-spectrograms is a common baseline that already yields high accuracy Tsalera et al. (2021). Researchers have also explored parallel multi-scale CNNs (processing multiple spectrogram resolutions), and CNN+RNN hybrids (CNN for feature extraction, followed by RNN or GRU to capture longer temporal context in an audio clip). One example is a CNN-BiLSTM hybrid that first uses CNN layers on spectrogram patches, then feeds the features into a bidirectional LSTM to classify acoustic scenes; such models have won DCASE challenges (Detection and Classification of Acoustic Scenes and Events). In DCASE 2019 Task 1 (acoustic scene classification), top systems often combined CNNs with attention or gated mechanisms to focus on salient sound events. Attention modules on top of CNN features help the model weight important sound segments – e.g. a loud car horn – more than background noise Gong et al. (2021). Sound Event Tagging: Beyond single-label classification, CNNs have been used for audio tagging (multi-label classification of simultaneous sound events). A prominent effort is PANNs (Pretrained Audio Neural Networks) by Kong et al. (2020), who trained multiple CNN architectures on the full AudioSet for general audio tagging Kong et al. (2020). Their largest model ("CNN14", a 14-layer CNN akin to VGG/ResNet) achieved mean average precision (mAP) = 0.439 on AudioSet, exceeding previous state-of-the-art (Google's earlier model had mAP 0.314). Notably, they introduced a Wavegram branch to learn from raw waveform in parallel with mel-spectrogram, improving tagging performance. PANNs' CNNs can be fine-tuned for specific tasks with limited data, a form of transfer learning for audio. This illustrates how large-scale CNN pretraining benefits even smaller ESC datasets.

3.7.3 Music Classification (Genres, Tags, and Audio Content)

Music classification covers tasks like music genre recognition, instrument classification, and automatic tagging of music attributes (mood, era, etc.). Compared to speech, music audio tends to have more complex harmonic structure and longer-term temporal dependencies (bars, beats). CNNs have been successfully applied to music classification by treating spectrograms of music audio as images, sometimes in combination with temporal models.

Early deep learning work (before 2015) on genre used fully connected nets or shallow CNNs on features; but by late 2010s, CNNs became state-of-the-art. For example, Kereliuk et al. (2015) and Choi et al. (2017) used CNNs on mel-spectrograms to classify genres and achieved significantly higher accuracy on the GTZAN genre dataset (10 genres) than traditional MFCC+SVM approaches. Dong et al. (2018) proposed a CNN that first divides a music clip into segments and extracts features, then aggregates them to predict the genre. With sufficient training data, CNNs can learn timbral textures that distinguish genres (e.g. the aggressive guitar spectra in rock vs. the smooth beats in jazz). On the FMA music dataset (a 2017 collection with 8 large-scale genres) Defferrard et al. (2017), 2D CNNs have achieved strong performance (60–70% accuracy on 8 genres, which is notable given the complexity and overlap of music genres).

One challenge in music is the scale of patterns: rhythmic patterns occur over seconds, while CNN receptive fields are usually much smaller. To address this, some models apply dilated convolutions or temporal pooling to capture wider context. Others use CRNNs – for example, Choi et al. (2017)

designed a Convolutional Recurrent Neural Network for music tagging that used CNN layers to capture local music features and an RNN to capture song-level temporal dependencies. This CRNN achieved state-of-art on the MagnaTagATune dataset (a multi-label music tagging set).

Music Auto-tagging and Representation Learning: Multi-label tagging (predicting all genres/moods present in a track) is more complex than single-genre classification. CNNs have been combined with attention mechanisms to attend to important sections of a song. For instance, Pons & Serra (2019) employed a CNN with learned filter shapes specialized for musical pitch vs. rhythm, improving instrument recognition. Self-supervised learning has also entered music analysis: music representation models (e.g. COLA – Contrastive Learning of Audio, 2020 Saeed et al. (2021)) often use a CNN encoder to learn from unlabeled music via contrastive losses, which can then be fine-tuned for genre or mood classification, yielding improvements especially when labeled data is scarce.

3.7.4 Speaker Identification and Voice Classification

Speaker identification (determining who is speaking from audio) and speaker verification (confirming if two clips are the same speaker) are tasks where CNNs have made substantial impact. Traditional speaker recognition relied on spectral features (MFCCs) and Gaussian mixture models or i-vectors; the trend shifted in the late 2010s toward neural embeddings (d-vectors, x-vectors) extracted by deep networks. CNNs, including 1D and 2D forms, are now a common choice for extracting speaker representations from audio.

A prominent approach is to train a CNN to classify a large number of training speakers, then use the penultimate layer as a fixed-dimensional speaker embedding (e.g. a 256-D vector) for any input utterance. Snyder et al. (2018) introduced the X-vector approach using a TDNN (time-delayed CNN) to produce speaker embeddings, which significantly improved speaker recognition benchmarks. Subsequent works replaced the TDNN with deeper CNNs or ResNets on spectrogram inputs. For example, VoxCeleb2 is a dataset of over 1 million utterances from 6112 speakers collected from YouTube Chung et al. (2018). models trained on VoxCeleb2 include VGG-type CNNs and ResNet-34 architectures that have achieved excellent performance on speaker verification (e.g. 2% equal error rate on VoxCeleb1 test). These CNNs operate on mel-spectrogram or MFCC inputs and learn to discriminate thousands of speaker identities, effectively learning voice signatures. At inference, the last-layer embeddings from the CNN are compared (using cosine similarity or PLDA) to decide if two utterances are from the same speaker. CNNs are well-suited here because they can learn fine spectral details (formant structures, pitch patterns, mic characteristics) that distinguish speakers, while being robust to time shifts and phonetic content.

As mentioned, SincNet was applied to speaker identification with notable success. By learning custom band-pass filters, SincNet's CNN achieved higher accuracy than standard CNNs on tasks like TIMIT speaker recognition Ravanelli & Bengio (2019). Other raw-wave models (e.g. SpeakerNet or WaveNet-based classifiers) have been tried, but most high-performing systems still feed CNNs with spectrograms, possibly augmented by acoustic features like per-channel energy normalization (PCEN) that dampens channel variability. The choice of architecture ranges from small-footprint CNNs for embedded speaker ID (to deploy on devices for voice login) to large ResNets with angular margin softmax losses to maximize discriminability.

4 Conclusion

Convolutional networks have journeyed from LeNet-5's hand-tuned filters to billion-parameter systems that learn their own spectro-temporal front ends, yet their core virtue—enforcing local coherence—remains constant. That inductive bias lets CNNs capture the short-time energy modulations that distinguish phonemes, bird calls, and percussive hits with far fewer parameters than attention-only models. When coupled with large-scale pre-training on AudioSet or YouTube-100M, these same networks transfer across speech, music, scene, and speaker tasks with a few minutes of fine-tuning data. Practical advances—depthwise separable kernels, mixed-precision training, and distilled student models—now deliver sub-millisecond inference on earbuds and edge sensors, making always-on acoustic intelligence feasible without cloud latency or privacy leakage.

Looking ahead, the frontier lies in blending the pattern-sniffing strength of convolution with the global context modeling of transformers and graph networks. Masked audio modeling, contrastive

pre-text tasks, and audio-visual co-training are rapidly shrinking the gap between supervised and self-supervised accuracy, while also taming domain shift and label scarcity. Yet three hurdles persist: robustness to out-of-distribution noise, principled uncertainty estimates for safety-critical settings, and human-interpretable explanations of what the filters "hear." Tackling these problems will require richer benchmarks that include adversarial perturbations, causal interventions, and perceptual studies alongside accuracy scores. If those challenges are met, CNN-centric hybrids could form the low-power backbone of ubiquitous sensing—alerting us to dangerous alarms, enriching immersive media, and giving voice to the next generation of ambient computing.

REFERENCES

Sanjeev Arora, Aditya Bhaskara, Rong Ge, and Tengyu Ma. Provable bounds for learning some deep representations, 2013. URL https://arxiv.org/abs/1310.6343.

John S. Bridle. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In Françoise Fogelman Soulié and Jeanny Hérault (eds.), *Neurocomputing*, pp. 227–236, Berlin, Heidelberg, 1990. Springer Berlin Heidelberg. ISBN 978-3-642-76153-9.

Keunwoo Choi, György Fazekas, Kyunghyun Cho, and Mark Sandler. Convolutional recurrent neural networks for music classification. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 2392–2396, 2017. doi: 10.1109/ICASSP.2017.7952132.

Joon Son Chung, Arsha Nagrani, and Andrew Zisserman. Voxceleb2: Deep speaker recognition. In *Interspeech 2018*, pp. 1086–1090, 2018. doi: 10.21437/Interspeech.2018-1929.

Dan Ciresan, Ueli Meier, Jonathan Masci, Luca Maria Gambardella, and Jürgen Schmidhuber. Flexible, high performance convolutional neural networks for image classification. pp. 1237–1242, 07 2011. doi: 10.5591/978-1-57735-516-8/IJCAI11-210.

Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2005)*, 2, 06 2005.

Michaël Defferrard, Kirell Benzi, Pierre Vandergheynst, and Xavier Bresson. FMA: A Dataset For Music Analysis. In *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2017. URL https://arxiv.org/abs/1612.01840. Camera-ready version; originally submitted as arXiv:1612.01840 (2016).

Zheng Dong, Wei-Chih Yeh, Chu Huang, and Jun Lee. A segment-based convolutional neural network for music genre classification. *IEEE Transactions on Multimedia*, 20(7):1544–1553, 2018. doi: 10.1109/TMM.2018.2839163.

Jort F. Gemmeke, Daniel P. W. Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R. Channing Moore, Manoj Plakal, and Marvin Ritter. Audio set: An ontology and human-labeled dataset for audio events. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 776–780, 2017. doi: 10.1109/ICASSP.2017.7952261.

Yuan Gong, Yu-An Chung, and James Glass. Ast: Audio spectrogram transformer. In *Interspeech 2021*, pp. 571–575, 2021. doi: 10.21437/Interspeech.2021-698.

Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. In *Proceedings of Interspeech*, pp. 5036–5040. ISCA, 2020. doi: 10.21437/Interspeech.2020-3015. URL https://doi.org/10.21437/Interspeech.2020-3015.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

Shawn Hershey, Sourish Chaudhuri, Daniel P. W. Ellis, Jort F. Gemmeke, Aren Jansen, R. Channing Moore, Manoj Plakal, Devin Platt, Rif A. Saurous, Bryan Seybold, Malcolm Slaney, Ron J. Weiss, and Kevin Wilson. Cnn architectures for large-scale audio classification. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 131–135, 2017. doi: 10.1109/ICASSP.2017.7952132.

Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors, 2012. URL https://arxiv.org/abs/1207.0580.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015. URL https://arxiv.org/abs/1502.03167.

Zachary Kereliuk, Conrad Sanderson, and Brian C. Lovell. Deep neural networks for music classification and tagging. In *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR)*, pp. 427–433, 2015.

Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley. PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28(11):2880–2894, 2020. doi: 10.1109/TASLP.2020.3030497.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger (eds.), *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012. URL https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.

Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

Jason Li, Vitaly Lavrukhin, Boris Ginsburg, Ryan Leary, Oleksii Kuchaiev, Jonathan M. Cohen, Huyen Nguyen, and Ravi Teja Gadde. Jasper: An end-to-end convolutional neural acoustic model. In *Interspeech 2019*, pp. 71–75, 2019. doi: 10.21437/Interspeech.2019-1819.

David G. Lowe. Distinctive image features from scale-invariant keypoints. *Int. J. Comput. Vision*, 60(2):91–110, November 2004. ISSN 0920-5691. doi: 10.1023/B:VISI.0000029664.99615.94. URL https://doi.org/10.1023/B:VISI.0000029664.99615.94.

Karol J. Piczak. ESC: Dataset for Environmental Sound Classification. In *Proceedings of the 23rd Annual ACM Conference on Multimedia*, pp. 1015–1018. ACM Press. ISBN 978-1-4503-3459-4. doi: 10.1145/2733373.2806390. URL http://dl.acm.org/citation.cfm?doid=2733373.2806390.

Jordi Pons and Xavier Serra. Randomly weighted cnns for (music) audio classification. In *ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 336–340. IEEE, 2019.

Mirco Ravanelli and Yoshua Bengio. Speech and speaker recognition from raw waveform with sincnet, 2019. URL https://arxiv.org/abs/1812.05920.

Aaqib Saeed, David Grangier, and Neil Zeghidour. Contrastive learning of general-purpose audio representations. In *ICASSP 2021 – 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 3875–3879. IEEE, 2021.

Dominik Scherer, Andreas Müller, and Sven Behnke. Evaluation of pooling operations in convolutional architectures for object recognition. In Konstantinos Diamantaras, Wlodek Duch, and Lazaros S. Iliadis (eds.), *Artificial Neural Networks – ICANN 2010*, pp. 92–101, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg. ISBN 978-3-642-15825-4.

Alok Sharma, Edwin Vans, Daichi Shigemizu, Keith A Boroevich, and Tatsuhiko Tsunoda. Deepinsight: A methodology to transform a non-image data to an image for convolution neural network architecture. *Scientific reports*, 9(1):11399, 2019.

Patrice Y. Simard, Dave Steinkraus, and John Platt. Best practices for convolutional neural networks applied to visual document analysis. Institute of Electrical and Electronics Engineers, Inc., August 2003. URL https://www.microsoft.com/en-us/research/publication/best-practices-for-convolutional-neural-networks-applied-to-visual-document-analysis/.

Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015. URL https://arxiv.org/abs/1409.1556.

David Snyder, Daniel Garcia-Romero, Gregory Sell, Daniel Povey, and Sanjeev Khudanpur. X-vectors: Robust dnn embeddings for speaker recognition. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5329–5333. IEEE, 2018. doi: 10.1109/ICASSP.2018.8461375.

Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions, 2014. URL https://arxiv.org/abs/1409.4842.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

Eleni Tsalera, Andreas Papadakis, and Maria Samarakou. Comparison of pre-trained cnns for audio classification using transfer learning. *Journal of Sensor and Actuator Networks*, 10(4), 2021. ISSN 2224-2708. doi: 10.3390/jsan10040072. URL https://www.mdpi.com/2224-2708/10/4/72.

Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. *Comput. Vis. Pattern Recog*, 1, 01 2001.

Khalid Zaman, Melike Sah, Cem Direkoglu, and Masashi Unoki. A survey of audio classification using deep learning. *IEEE Access*, 11:106620–106649, 2023. doi: 10.1109/ACCESS.2023.3318015.